

Úvod do databázových systémů

Kapitola 9
Jazyk SQL (2)

Úvod

- příkaz SELECT

- formulace dotazů nad tabulkou

- de facto implementace relační algebry
 - vstupem jsou tabulky (tj. v podstatě relace)
 - výstupem je opět tabulka (tj. v podstatě relace)
 - rozdíl mezi tabulkou a relací viz předchozí přednášky
 - pořadí řádků výsledku není zaručeno (bez zadání ORDER BY)

- výraz v SQL nedefinuje způsob provedení dotazu

- SŘBD provádí optimalizaci provádění dotazu
 - rozdíl výraz vs. dotaz viz předchozí přednášky

Úvod

- blok SELECT-FROM-WHERE (zjednodušeně)

SELECT col1, col3
FROM table_name
WHERE condition;

jaké sloupce chceme zobrazit

z jakých tabulek

na základě jaké podmínky

- obecněji (s přepisem do relační algebry)

SELECT $A_1, A_2, \dots, A_n$
FROM $R_1, R_2, \dots, R_m$
WHERE φ

$(R_1 \times R_2 \times \dots \times R_m)(\varphi) [A_1, A_2, \dots, A_n]$

- neexistence podmínky se chápe jako TRUE

Úvod

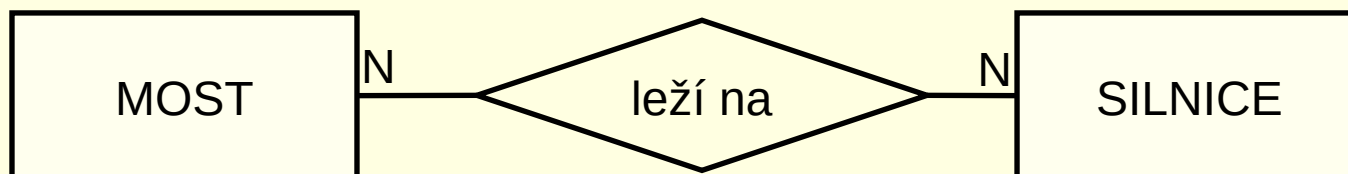
- blok SELECT-FROM-WHERE

- přepis do relační algebry není zcela ekvivalentní, neboť SQL má oproti RMD některé odlišnosti
 - standardní SELECT neodstraňuje z výsledku duplicitní n-tice (tj. výsledek nemusí být relace)
 - to se zařídí klauzulí **DISTINCT** hned za slovem SELECT
 - SQL uvažuje prázdné hodnoty, tj. tříhodnotová logika apod.
- chceme-li vybrat všechny sloupce, místo jejich výčtu lze použít hvězdičku *
- chceme-li výsledek uspořádat, přidáme na konec

ORDER BY col_name [{ASC | DESC}]

Schema použité v příkladech

- máme databázi silnic a mostů, které na nich leží
 - na jedné silnici může stát víc mostů
 - jeden most může stát *de iure* na více silnicích
 - určitý úsek silnice může být očíslován více čísly
 - atributy silnice
 - číslo, třída, start, cíl, pověřený vlastník, délka, datum poslední opravy
 - atributy mostu
 - číslo, nosnost, šířka, technologie



Schema použité v příkladech

■ Silnice

<u>číslo s</u>	třída	start	cíl	vlastník	délka	oprava
21	1	D5	Německo	stát	62	12.6.2009
53	1	Znojmo	Pohořelice	stát	38	30.5.2007
230	2	Nepomuk	Bečov nad Teplou	Plzeňský kraj	88	14.2.1999
276	2	Bělá pod Bezdězem	Kněžmost	Liberecký kraj	21	21.8.2003
403	2	Kouty	Telč	kraj Vysočina	36	2.10.2001
602	2	Pelhřimov	Starý Lískovec	kraj Vysočina	108	5.8.2008
0395	3	Kostelec	Cejle	kraj Vysočina	2,5	28.9.1992
1314	3	Štoky	Smrčná	kraj Vysočina	6,3	10.7.2000

Schema použité v příkladech

■ Most

<u>číslo m</u>	nosnost	šířka	technologie
2469	55	11	beton
8963	40	8	beton
8965	42	9	beton
500489	3,5	4	kámen

■ Leží_na

<u>číslo m</u>	<u>číslo s</u>
2469	21
2469	230
8963	602
8965	602
500489	0395

Jednoduché dotazy

■ příklady

- vypiš celou tabulku Silnice

```
SELECT * FROM Silnice;
```

- čísla a délky silnic 1. a 2. třídy na Vysočině

```
SELECT číslo_s, délka  
FROM Silnice  
WHERE vlastník = 'kraj Vysočina'  
AND třída < 3;
```

číslo_s	délka
403	36
602	108

- čísla silnic s mostem, v prvním sloupci slovo „Číslo:“, každé číslo silnice vypsát jen jednou

```
SELECT DISTINCT 'Číslo:', číslo_s  
FROM Leží_na;
```

'Číslo:'	číslo_s
Číslo:	21
Číslo:	230
Číslo:	602
Číslo:	0395

Jednoduché dotazy

■ příklady

- technologie mostů s nosností < 50 t

```
SELECT technologie
```

```
FROM Most
```

```
WHERE nosnost < 50;
```

technologie
beton
beton
kámen

- technologie mostů s nosností < 50 t, **každou použitou technologii vypiš jen jednou**, **seřaď sestup**.

```
SELECT DISTINCT technologie
```

```
FROM Most
```

```
WHERE nosnost < 50
```

```
ORDER BY technologie DESC;
```

technologie
kámen
beton

Jednoduché dotazy

■ příklady

- čísla a nosnosti mostů, jež jsou široké 8 až 12 m

```
SELECT číslo_m AS 'číslo mostu', nosnost  
FROM Most  
WHERE šířka >= 8 AND šířka <= 12;
```

číslo mostu	nosnost
8963	40
8965	42
2469	55

- totéž

```
SELECT číslo_m AS 'číslo mostu', nosnost  
FROM Most  
WHERE šířka BETWEEN 8 AND 12;
```

- doplněk

```
SELECT číslo_m AS 'číslo mostu', nosnost  
FROM Most  
WHERE šířka NOT BETWEEN 8 AND 12;
```

Dotazy nad více tabulkami

■ příklady

- najdi čísla silnic, na nichž leží nějaký most
- stačí jediná tabulka – všechny silnice s mosty jsou totiž evidovány ve vztahové tabulce Leží_na

```
SELECT číslo_s  
FROM Leží_na;
```

číslo_s
21
230
602
602
0395

Dotazy nad více tabulkami

■ příklady

- najdi čísla a délky silnic, na nichž leží nějaký most
- tečková notace odkazů na použité tabulky

```
SELECT číslo_s, délka  
FROM Silnice S, Leží_na L  
WHERE S.číslo_s = L.číslo_s
```

alias a jeho použití

číslo_s	délka
21	62
230	88
602	108
602	108
0395	2,5

Dotazy nad více tabulkami

- struktura použitých tabulek je následující:
 - tabulka Leží_na je vztahová tabulka, reprezentující vztah M:N mezi silnicí a mostem
 - cizími klíči odkazuje jednak na tabulku Silnice (na číslo_s), jednak na tabulku Most (na číslo_m)
 - podíváme-li se do sloupce číslo_s v tabulce Leží_na, budou v něm uložena čísla všech silnic, na kterých leží nějaký most
 - pokud na nějaké silnici leží mostů více, bude se číslo silnice ve sloupci opakovat tolikrát, kolik mostů na dané silnici leží
 - za každý výskyt vztahu most-silnice musí být v tabulce jeden záznam

Dotazy nad více tabulkami

- uvedený příklad tedy pracuje takto:
 - najde v tabulce Silnice takové záznamy, k nimž existuje odpovídající záznam v tabulce Leží_na
 - z vybraných záznamů pak zobrazí pouze sloupce číslo_s a délka.
- přesněji
 - nejprve se vytvoří kartézský součin tabulky Silnice a Leží_na
 - poté se z kombinovaných řádků vyberou takové, u nichž platí, že číslo_s původem z tabulky Silnice a číslo_s původem z tabulky Leží_na jsou stejná

Dotazy nad více tabulkami

- jak by se uvedeného docílilo v relační algebře?
 - můžeme pochopitelně také využít kartézského součinu a následné selekce s projekcí

$(\text{Silnice} \times \text{Leží_na})(S.\text{číslo_s} = L.\text{číslo_s})[S.\text{číslo_s}, \text{délka}]$

- výhodnější je ale použít operaci **spojení** – v tomto případě přirozené spojení

$(\text{Silnice} * \text{Leží_na})[\text{číslo_s}, \text{délka}]$

- příklad vlastně v SQL realizoval operaci přirozeného spojení, tj. speciální případ spojení přes rovnost: onu rovnost odpovídajících si atributů v SQL zápisu hezky vidíme

Dotazy nad více tabulkami

- příklad

- najdi šířky mostů v cm, které leží na silnicích první třídy
- aritmetika

```
SELECT M.šířka * 100
FROM Most M, Silnice S, Leží_na L
WHERE
    S.číslo_s = L.číslo_s
    AND M.číslo_m = L.číslo_m
    AND S.třída = 1
```


Dotazy nad více tabulkami

- jak to pracuje
 - nejprve se podíváme, ve kterých tabulkách se nalézají požadované informace:
 - šířky mostů jsou v tabulce Most
 - třídy silnic jsou v tabulce Silnice
 - který most na které silnici leží najdeme v Leží_na
 - při řešení úlohy musíme tedy sáhnout do všech tří tabulek
 - v relační algebře bychom opět použili spojení, selekci a projekci:

$(\text{Silnice} * \text{Leží_na} * \text{Most})(\text{třída}=1)[\text{šířka}]$

Spojení

- jedna z nejdůležitějších operací používaných v databázi
- umožňuje integrovat data z více tabulek, nejčastěji z těch, mezi nimiž je nějaký vztah
- ve starších verzích SQL bylo nutné spojení zapisovat explicitně pomocí kartézského součinu a následné selekce za klauzulí WHERE (viz příklad)
- novější verze SQL (ne však všechny SŘBD) umožňují i elegantnější zápis spojení za klauzulí FROM
- spojení všech možných druhů: přirozené, theta-spojení (přes podmínku), vnější spojení

Spojení

- implementace spojení nejrůznějšího druhu
 - pomocí dosud probraných konstruktů
 - vnitřní Θ -spojení nebo přirozené spojení zkonstruujeme jako omezený kartézský součin

```
SELECT ... FROM tbl1, tbl2 WHERE tbl1.A = tbl2.B
```
 - levé/pravé polospojení se definuje výběrem sloupců za SELECT
 - nové konstrukty SQL92
 - kartézský součin

```
... FROM tbl1 CROSS JOIN tbl2 ...
```
 - přirozené spojení

```
... FROM tbl1 NATURAL JOIN tbl2 ...
```

Spojení

- implementace spojení nejrůznějšího druhu

- spojení přes podmínku

- ... FROM tbl1 JOIN tbl2 ON A<B ...

- levé/pravé/plné **vnější** spojení

- ... FROM tbl1 NATURAL OUTER JOIN tbl2 ...

- ... FROM tbl1 {LEFT | RIGHT | FULL}

- {OUTER | INNER } JOIN tbl2 ON cond ...

Spojení

- PostgreSQL

- vnější spojení se předpokládá, kdykoli se použije slovo LEFT, RIGHT nebo FULL

- tj. ... FROM tbl1 LEFT JOIN tbl2 ...

- je stejné jako ... FROM tbl1 LEFT OUTER JOIN tbl2 ...

- analogicky pro ostatní vnější spojení

Spojení

- PostgreSQL

- existuje ještě konstrukt **USING**

- ... FROM tbl1 JOIN tbl2 USING(a, b)
 - což je stejné jako

... FROM tbl1 JOIN tbl2
ON tbl1.a = tbl2.a AND tbl1.b = tbl2.b ...

- USING ovšem do výsledku pustí jen jednu kopii sloupce **a** a jednu kopii sloupce **b**, ne obě

Spojení

- příklad (pokračování)
 - najdi čísla a délky silnic, na nichž leží nějaký most
 - bez spojení

```
SELECT číslo_s, délka  
FROM Silnice S, Leží_na L  
WHERE S.číslo_s = L.číslo_s
```

číslo_s	délka
21	62
230	88
602	108
602	108
0395	2,5

Spojení

- příklad (pokračování)

- najdi čísla a délky silnic, na nichž leží nějaký most
- se spojením

```
SELECT číslo_s, délka  
FROM Silnice NATURAL JOIN Leží_na;
```

- nebo

```
SELECT číslo_s, délka  
FROM Silnice JOIN Leží_na USING(číslo_s)
```

- nebo

```
SELECT číslo_s, délka  
FROM Silnice S JOIN Leží_na L ON S.číslo_s=L.číslo_s;
```

alias a jeho použití

Spojení

- příklad (pokračování)
 - najdi šířky mostů v cm, které leží na silnicích první třídy
 - bez spojení

```
SELECT M.šířka * 100
FROM Most M, Silnice S, Leží_na L
WHERE
    S.číslo_s = L.číslo_s
    AND M.číslo_m = L.číslo_m
    AND S.třída = 1
```

Spojení

- příklad (pokračování)

- najdi šířky mostů v cm, které leží na silnicích první třídy
- se spojením

```
SELECT šířka * 100
FROM Most NATURAL JOIN Leží_na NATURAL JOIN
Silnice
WHERE třída = 1;
```

- při spojování více tabulek je třeba dbát na správné pořadí tabulek, jistota je ohraničit každou spojovanou dvojici závorkami

Spojení

- nové konstrukty pro spojení
 - podstatně zvyšují čitelnost a srozumitelnost kódu
 - jazyk SQL se více přiblížil relační algebře
 - nyní má tedy znalost relační algebry ještě větší opodstatnění, neboť je to formální popis operací nezávislý na implementaci
- v praxi je podstatné především přirozené spojení
 - pokud při definici cizích klíčů striktně dodržujeme zásadu, že se cizí klíč i primární klíč odkazované tabulky **jmenují stejně**, pak nám přirozené spojení stačí ve většině situací
 - pokud se **nejmenují stejně**, NATURAL JOIN vrátí kartézský součin – to je chyba, která se velmi obtížně detekuje!

Přepis do relační algebry

■ přirozené spojení

```
SELECT A1, A2, ..., An  
FROM R1 NATURAL JOIN R2 NATURAL JOIN R3  
WHERE  $\varphi$ 
```

$$(R_1 * R_2 * R_3)(\varphi) [A_1, A_2, \dots, A_n]$$

■ Θ -spojení

```
SELECT A1, A2, ..., An  
FROM R1 JOIN R2 ON (R1.X  $\Theta$  R2.Y)  
WHERE  $\varphi$ 
```

$$(R_1 [R_1.X \Theta R_2.Y] R_2) (\varphi) [A_1, A_2, \dots, A_n]$$